



Informe: Regresiones en Machine Learning



Menú de contenidos

- Regresiones:
 - Lineal
 - Cuadrática
 - Polinomial
 - Casos relacionados: Logarítmica y Exponencial
 - Dos o más variables
 - Regresiones con “esteroides” (Regularización y Descenso del Gradiente)
 - Cuando conocemos la relación funcional: *Curve Fit*
 - Regresión Lineal Bayesiana (*próxima clase*)
-

1. Introducción a las Regresiones

La **regresión** busca modelar la relación entre una variable independiente (**X**) y una variable dependiente (**Y**).

Ejemplo básico:

$$Y = a \cdot X + b$$

- **X** → variable independiente (explicativa o regresora).
- **Y** → variable dependiente (respuesta).
- **a, b** → parámetros a estimar a partir de los datos.

En la vida real la relación no es perfecta, por lo que se incluye un **residuo** (r_i):

$$y_i = a \cdot x_i + b + r_i$$

✓ Idea clave

- Queremos que los **residuos** sean lo más pequeños posibles.
- Para medirlos, se utiliza el **Error Cuadrático Medio (ECM)**:

$$ECM = \frac{1}{n} \sum (r_i^2)$$

- El objetivo es encontrar los valores de **a** y **b** que **minimicen el ECM**.
 - Este procedimiento se llama **Método de Mínimos Cuadrados**.
-

2. Herramientas en Python

Existen varias bibliotecas para implementar regresiones:

Scikit-learn

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, y)
```

- [!\[\]\(33006de4dd11f8c729ca8ca0fde0352f_img.jpg\) Documentación](#)

Statsmodels

```
import statsmodels.api as sm
model = sm.OLS(y, X).fit()
print(model.summary())
```

- [!\[\]\(adb0331d22f78481623cc605df40612a_img.jpg\) Ejemplos](#)

- **Scipy** (Least Squares & Curve Fit)
 - 📖 `curve_fit`
 - 📖 `lsq_linear`
-

3. Tipos de Regresiones Lineales Avanzadas

- **Ridge Regression (L2)** → penaliza parámetros grandes.
- **Lasso Regression (L1)** → fuerza a que algunos coeficientes sean exactamente 0.
- **Elastic Net (L1 + L2)** → combinación de Ridge y Lasso.
- **SGD (Stochastic Gradient Descent)** → usa descenso de gradiente en lugar de mínimos cuadrados.
- **Regresiones Robustas** → resistentes a *outliers*.

🔗 Enlaces:

- [Ridge & Lasso en sklearn](#)
 - [Regresiones robustas](#)
-

4. Regresiones Polinomiales

Ejemplo de función:

$$Y = a_0 + a_1X + a_2X^2 + a_3X^3 + \dots$$

👉 Aunque parezca no lineal, se puede resolver con **regresión lineal** aplicando *PolynomialFeatures*.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)
```

5. Regresiones Logarítmicas y Exponenciales

Algunos problemas pueden “linealizarse”:

- Exponencial:

$$Y = a \cdot e^{bX} \Rightarrow \ln(Y) = \ln(a) + bX$$

- Logarítmica:

$$Y = a + b \ln(X)$$

En estos casos, se puede usar **curve_fit** de Scipy para ajustar la función conocida.

 [Documentación curve_fit](#)

6. Regresiones No Lineales

Ejemplo de modelo:

$$Y = a_0 \cdot X^{a_1}$$

Estos modelos se ajustan con métodos iterativos y optimización numérica.

7. Regresiones con Más de una Variable

También llamadas **regresiones múltiples**.

Ejemplo:

$$Y = a_0 + a_1X_1 + a_2X_2 + \dots + a_nX_n$$

 Ejemplo práctico en Google Colab:

 Ver Notebook

Conclusión

- La regresión es una herramienta fundamental para predecir y entender relaciones entre variables.
- Existen múltiples variantes según la naturaleza del problema: lineales, polinomiales, robustas, logarítmicas, exponenciales y no lineales.
- En Python, las librerías **Scikit-learn**, **Statsmodels** y **Scipy** cubren prácticamente todos los casos.